VOLUME II · DRILL PACK · MOCK PAPER · WALK-IN CHEATSHEET

CMS 704 — Practice, Worked Drills & Final Recall

The companion to the Volume I study guide. This paper contains real calculation, so this volume is built around it: a full mock paper with model answers, then every calculable topic drilled with the arithmetic shown and independently checked.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 05 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

HOW TO USE THIS PACK

TIME YOU HAVE	DO THIS
3 hours	Sit the mock paper (§1) closed-book → mark against §2 → re-drill only what you lost marks on (§3–§7) → §9 traps → sleep.
90 minutes	Conversion and signed-number drills (§3, §4) → minimization and K-map drills (§5) → RTL drills (§6) → Iron Law drills (§7) → §9 traps.
45 minutes	Rapid-fire recall (§8) covering the answer column → §9 traps → §10 walk-in sheet.
10 minutes at the door	§10 only.

Rule for a calculation paper: write the **formula**, then the **substitution**, then the **answer** — three lines, every time. A slip in the last line still leaves two lines of method marks standing. A bare answer with no working earns nothing even when it is right.

1 Mock paper — sit this closed-book

RIVERS STATE UNIVERSITY · PGD COMPUTER SCIENCE · CMS 704 — COMPUTER ARCHITECTURE
Mock examination · Time allowed: 3 hours · **Answer any FIVE questions** · Each question carries 20 marks · All working must be shown

Q	QUESTION	MARKS
1	**(a)** Define computer architecture and computer organization. **(b)** Using a table, state **five** differences between them. **(c)** State any **four** things each of them specifies.	4 + 10 + 6
2	**(a)** What is a logic gate? **(b)** Draw the symbol and write the boolean expression for the AND, OR, NOT, NAND, NOR, XOR and XNOR gates. **(c)** Given two inputs A and B, construct a truth table showing `A·B`, `A+B`, `(A·B)'` and `(A+B)'`. **(d)** How many rows does a truth table with **n** inputs have? Hence give the number of rows for three inputs.	3 + 7 + 6 + 4
3	**(a)** What is logic expression minimization, and why is it important? Give three reasons. **(b)** State the seven laws of boolean algebra in both their AND and OR forms. **(c)** A majority vote circuit takes inputs A, B and C. Construct its truth table, write the sum-of-products expression and minimize it algebraically. **(d)** Using a Karnaugh map, minimize `F(A,B,C) = Σm(1,3,5,7)`.	5 + 5 + 6 + 4
4	**(a)** Define Register Transfer Language and explain any four of its symbols. **(b)** State and explain the four types of micro-operation, giving one example of each. **(c)** Write RTL statements for: (i) if K = 1 transfer R1 to R2, otherwise transfer R4 to R2; (ii) if Z = 1 and C = 0, increment the program counter; (iii) transfer R1 into R2 and R3 into R4 in the same clock cycle.	7 + 8 + 5
5	**(a)** Convert **2025_{10}** to base 2, base 8 and base 16, showing your working. **(b)** Convert **754_8** to base 10 and to base 2. **(c)** State the relationship between base 8 and base 2, and between base 16 and base 2, and use it to convert 1101101_2 to octal and hexadecimal. **(d)** Represent **−45** as an 8-bit number in signed magnitude, one's complement and two's complement. **(e)** State the range of an 8-bit unsigned number.	6 + 3 + 4 + 5 + 2
6	**(a)** With the aid of a diagram, describe the memory hierarchy and state the speed, capacity and purpose of each level. **(b)** Define cache memory and distinguish between a cache hit and a cache miss. **(c)** What is locality of reference? Explain its two main types with an example each. **(d)** State the CPU performance equation and use it to find the CPU time of a program of 5×10^8 instructions with a CPI of 2 running on a 2.5 GHz processor.	6 + 4 + 5 + 5

2 Model answers — the marks-earning shape

Q1 · architecture vs organization

Open with both definitions (Volume I §1), then the table with a **basis of comparison** column: what it answers (what vs how) · level (specification vs implementation) · visible to (programmer/compiler vs hardware engineer) · stability (constant across implementations vs varies between models) · driven by (functional requirements vs cost, power and size trade-offs). Close with the x86-64 example: *software compiled for x86-64 runs on any processor implementing that architecture, regardless of internal implementation.* For (c) quote any four from each seven-item list.

Q2 · gates and truth tables

A **logic gate** is the fundamental building block of digital circuits — a device that performs a boolean function on one or more binary inputs and produces a single binary output. Draw all seven symbols with their expressions, then the four-row table (Volume I §3). Rows = $\mathbf{2^n}$, so three inputs give $\mathbf{2^3 = 8}$ **rows**. Two marks are usually reserved for saying the bubble on an output **is** the NOT — it converts AND→NAND, OR→NOR, XOR→XNOR.

Q3 · minimization CALCULATION

```
(c)  A B C │ F         F = A'BC + AB'C + ABC' + ABC
     0 0 0 │ 0
     0 0 1 │ 0         A'BC + ABC ⇒ BC(A'+A) = BC
     0 1 0 │ 0         ABC' + ABC ⇒ AB(C'+C) = AB
     0 1 1 │ 1 ①       AB'C + ABC ⇒ AC(B'+B) = AC
     1 0 0 │ 0
     1 0 1 │ 1 ②       ∴ F = BC + AB + AC
     1 1 0 │ 1 ③
     1 1 1 │ 1 ④

(d)  Σm(1,3,5,7) — minterms 1,3,5,7 are exactly the rows where C = 1

           BC
      A \  00  01  11  10
        0 │ 0 │ 1 │ 1 │ 0 │      the four 1s form one group of 4
        1 │ 0 │ 1 │ 1 │ 0 │      A changes, B changes, C stays 1

     ∴ F = C
```

Q4 · RTL

```
(c)(i)   K:  R2 ← R1          (ii)  Z·C': PC ← PC + 1     (iii)  R2 ← R1, R4 ← R3
         K': R2 ← R4
```

Say why in each case: the **else branch takes the complement** of the control signal; **AND of conditions is written as a product**; the **comma means the transfers happen simultaneously in one clock cycle**.

Q5 · number systems CALCULATION

```
(a) 2025_10

 2│2025          8│2025         16│2025
 2│1012 — 1      8│ 253 — 1     16│ 126 — 9
 2│ 506 — 0      8│  31 — 5     16│   7 — 14 (E)
 2│ 253 — 0      8│   3 — 7       │   0 —  7
 2│ 126 — 1      8│   0 — 3
 2│  63 — 0
 2│  31 — 1      = 3751_8         = 7E9_16
 2│  15 — 1
 2│   7 — 1      Check: 3×512 + 7×64 + 5×8 + 1 = 2025 ✓
 2│   3 — 1             7×256 + 14×16 + 9      = 2025 ✓
 2│   1 — 1
   │   0 — 1

= 11111101001_2   Check: 1024+512+256+128+64+32+8+1 = 2025 ✓
```

```
(b) 754_8 → base 10 and base 2
    7×8^2 + 5×8^1 + 4×8^0 = 448 + 40 + 4 = 492_10
    7 = 111, 5 = 101, 4 = 100  →  111101100_2

(c) 8 = 2^3 → 3 : 1         16 = 2^4 → 4 : 1

    1101101_2 → 1 101 101 → 155_8     (64+40+5 = 109 ✓)
    1101101_2 → 0110 1101 → 6D_16     (96+13   = 109 ✓)

(d) −45 as an 8-bit number:   45_10 = 00101101_2

    Signed magnitude : 1 0101101  →  10101101_2
    One's complement : invert all →  11010010_2
    Two's complement : invert +1  →  11010011_2

(e) 8-bit unsigned range = 0 … 2^8 − 1 = 0 … 255
```

Q6 · memory and performance

```
(d)  CPU Time = Instruction Count × CPI × Clock Cycle Time
     Clock cycle time = 1 / 2.5×10^9 = 0.4 ns
     CPU Time = 5×10^8 × 2 × 0.4×10^-9 = 0.4 seconds
```

For (a) draw the pyramid with all five levels labelled and the two arrows (cost per bit rising upward; capacity and access time rising downward), then the table. For (b) and (c) use the Volume I §10 wording — cache hit retrieves in nanoseconds, cache miss forces a fetch from slower RAM and the block is copied in for future use; temporal locality is the loop or counter, spatial locality is the array.

3 Conversion drills — every answer double-checked

#	CONVERT	ANSWER	CHECK
1	**2025_{10}** → base 2	11111101001_2	1024+512+256+128+64+32+8+1 = 2025 ✓
2	**2025_{10}** → base 8	3751_8	3×512 + 7×64 + 5×8 + 1 = 2025 ✓ — or group the binary in 3s: `11 111 101 001`
3	**2025_{10}** → base 16	$7E9_{16}$	7×256 + 14×16 + 9 = 2025 ✓ — or group the binary in 4s: `0111 1110 1001`
4	**1101101_2** → 10, 8, 16	109_{10} · 155_8 · $6D_{16}$	64+32+8+4+1 = 109 · 64+40+5 = 109 · 96+13 = 109 ✓
5	**$3F5_{16}$** → 10, 2, 8	1013_{10} · 1111110101_2 · 1765_8	3×256 + 15×16 + 5 = 1013 · 512+448+48+5 = 1013 ✓
6	**754_8** → 10, 2, 16	492_{10} · 111101100_2 · $1EC_{16}$	448+40+4 = 492 · 256+224+12 = 492 ✓
7	**0.6875_{10}** → base 2	0.1011_2	×2 → 1.375 (1) · 0.75 (0) · 1.5 (1) · 1.0 (1). Back: ½ + ⅛ + 1/16 = 0.6875 ✓
8	**4832_{10}** → 2, 8, 16 CLASS	1001011100000_2 · 11340_8 · $12E0_{16}$	4096+512+128+64+32 = 4832 ✓

THE FOUR-STEP ROUTINE THAT NEVER FAILS

1. Decimal → any base: **divide repeatedly, read the remainders upwards. 2.** Any base → decimal: **expand positionally**, base ^ position, counting from 0 at the right. **3.** Between 2, 8 and 16: **never divide** — go through binary and group in **3s for octal**, **4s for hexadecimal**, padding the left group with zeros. **4.** Fractions: **multiply repeatedly by the base and read the integer parts downwards**.

Arithmetic drills

BINARY

```
  101101_2  (45)
+  11011_2  (27)
----------
 1001000_2  (72) ✓

    1101_2  (13)
×    101_2  ( 5)
----------
    1101
   0000
+ 1101
----------
 1000001_2  (65) ✓
```

OCTAL ADDITION

```
  507_8   (327)
+ 264_8   (180)
-------
  773_8   (507) ✓

Column by column:
7+4 = 11 → 11−8 = 3, carry 1
0+6+1 = 7
5+2 = 7
```

Carry whenever a column reaches **8**, not 10.

OCTAL SUBTRACTION

```
  642_8   (418)
− 375_8   (253)
-------
  245_8   (165) ✓

  246_8   (166)   ← class example
− 127_8   ( 87)
-------
  117_8   ( 79) ✓
```

When you borrow, you borrow **8**.

4 Signed-number drills

VALUE	MAGNITUDE IN BINARY (8-BIT)	SIGNED MAGNITUDE	ONE'S COMPLEMENT	TWO'S COMPLEMENT
−45	00101101	10101101	11010010	11010011
−100	01100100	11100100	10011011	10011100
−56 CLASS	0111000 (7-bit)	1111000	1000111	1001000
−127 CLASS	01111111	11111111	10000000	10000001
−1	00000001	10000001	11111110	11111111

The three-step routine. 1. Convert the magnitude to binary and pad to the stated width. **2. Signed magnitude** — flip the MSB to 1, leave the rest. **3. One's complement** — invert every bit. **4. Two's complement** — invert every bit, then add 1. Positive numbers are identical in all three.

RANGES TO QUOTE

SCHEME (N BITS)	RANGE	N = 8
Unsigned	$0 \dots 2^n - 1$	0 … 255
Signed magnitude	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127
One's complement	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127

```
-45 step by step

45   = 00101101
SM   = 1 0101101        ← MSB only
1's  = 11010010         ← invert all 8 bits
2's  = 11010010 + 1
     = 11010011
```

SCHEME (N BITS)	RANGE	N = 8
Two's complement	$-2^{n-1} \ldots +2^{n-1} - 1$	−128 … +127

Why two's complement wins in real hardware: it has only **one** representation of zero, so it gains one extra negative value, and ordinary binary addition works on it unchanged — no separate subtract circuit is needed. Signed magnitude and one's complement both waste a code on **−0**.

5 Minimization and K-map drills

#	SIMPLIFY	ANSWER	WORKING
1	`A'B'C + A'BC + AB'C + ABC`	**C**	Factor C: `C(A'B' + A'B + AB' + AB)`. The bracket covers every combination of A and B, so it equals 1, and `C·1 = C`.
2	`A'BC' + A'BC + ABC' + ABC`	**B**	`A'B(C'+C) = A'B` and `AB(C'+C) = AB`; then `A'B + AB = B(A'+A) = B`.
3	`AB + A(B+C) + B(B+C)`	**B + AC**	Expand: `AB + AB + AC + B + BC`. Idempotent kills the repeat; absorption gives `B + BC = B` and `B + AB = B`. Left with `B + AC`.
4	`(A + B'C)'`	**A'B + A'C'**	De Morgan: `A'·(B'C)' = A'·(B + C')`, then distribute → `A'B + A'C'`.
5	`A + A'B`	**A + B**	Distributive: `(A+A')(A+B) = 1·(A+B) = A+B`. A standard identity worth memorising.
6	`AB + AB'`	**A**	`A(B+B') = A·1 = A` — the complement law, which is what every K-map grouping is doing underneath.

K-MAP DRILL 1 — ΣM(0,1,4,5)

```
     BC
 A \  00  01  11  10
   0 | 1 | 1 | 0 | 0 |
   1 | 1 | 1 | 0 | 0 |

Group of 4 (the two left columns).
A changes, C changes, B stays 0.

∴ F = B'
```

K-MAP DRILL 2 — ΣM(2,3,6,7)

```
     BC
 A \  00  01  11  10
   0 | 0 | 0 | 1 | 1 |
   1 | 0 | 0 | 1 | 1 |

Group of 4 (the two right columns).
A changes, C changes, B stays 1.

∴ F = B
```

K-MAP DRILL 3 — ΣM(1,3,5,7)

```
     BC
 A \  00  01  11  10
   0 | 0 | 1 | 1 | 0 |
   1 | 0 | 1 | 1 | 0 |

Group of 4 (the two middle columns).
A changes, B changes, C stays 1.

∴ F = C
```

Column order is a mark on its own. The top of a 3-variable map reads **00 · 01 · 11 · 10**. If you write 00 · 01 · 10 · 11 the adjacent cells no longer differ by one bit, your groups become invalid, and the whole answer collapses even though the arithmetic inside it is fine. The same applies to a 4-variable map on both axes.

6 RTL drills — both directions

ENGLISH → RTL

STATEMENT	RTL
Transfer the contents of R3 into R1	`R1 ← R3`
Add R1 and R2, result into R3	`R3 ← R1 + R2`
Bitwise AND of R1 and R2 into R1	`R1 ← R1 ∧ R2`
If P = 1, move R1 into R2	`P: R2 ← R1`
If K = 1 move R1 into R2, else move R4	`K: R2 ← R1` `K': R2 ← R4`
If Z = 1 and C = 0, increment the PC	`Z·C': PC ← PC + 1`
Move R1 to R2 and R3 to R4 together	`R2 ← R1, R4 ← R3`
Load the low 8 bits of R1 into R2	`R2 ← R1(0–7)`

RTL → ENGLISH (THE REVERSE QUESTION)

RTL	WHAT IT SAYS
`T₀: MAR ← PC`	At time step T_0, the address in the program counter is copied into the memory address register — the start of the fetch.
`R2 ← R2 + 1`	An arithmetic micro-operation incrementing R2 by one.
`X'Y: R3 ← R1 ⊕ R2`	A logic micro-operation: only when X = 0 and Y = 1, R3 receives the bitwise XOR of R1 and R2.
`R1 ← R2, R2 ← R1`	The two registers **exchange** contents — legal only because the comma means both happen simultaneously in one clock cycle.

STATEMENT	RTL
R1 drives the bus; R2 latches from it	`Bus ← R1` then `R2 ← Bus`

Marking hints. The arrow always points **at the destination**. The condition goes **before** the colon, the operation after. An "else" is written with the **complemented** control signal on a second line, never as the word "else".

7 Performance drills

#	PROBLEM	ANSWER	WORKING
1	IC = 5×10^8, CPI = 2, clock = 2.5 GHz. Find the CPU time.	**0.4 s**	Cycle time = $1/2.5\times10^9$ = 0.4 ns. CPU time = $5\times10^8 \times 2 \times 0.4\times10^{-9}$ = 0.4 s
2	Same program — find the MIPS rating.	**1250 MIPS**	MIPS = IC / (exec time × 10^6) = 5×10^8 / (0.4 × 10^6) = 1250
3	IC = 2×10^9, CPI = 1.5, clock = 3.0 GHz. Find the CPU time.	**1.0 s**	Cycle = 0.333 ns; $2\times10^9 \times 1.5 \times 0.333\times10^{-9}$ = 1.0 s
4	A program of 10^9 instructions runs in 0.75 s on a 3 GHz machine. Find the CPI.	**2.25**	Rearrange: CPI = (CPU time × frequency) / IC = (0.75 × 3×10^9) / 10^9 = 2.25
5	Machine A runs a task in 8 s, Machine B in 12 s. Which is faster and by how much?	**A, by 1.5×**	Relative performance = Exec B / Exec A = 12/8 = 1.5
6	An optimisation cuts CPI from 2.0 to 1.6 with IC and clock unchanged. Find the speedup.	**1.25×**	CPU time ∝ CPI, so speedup = 2.0/1.6 = 1.25
7	A compiler halves the instruction count but doubles the CPI. What happens to performance?	**Unchanged**	CPU time = IC × CPI × cycle time; ½ × 2 = 1. The classic Iron Law trap.

8 Rapid-fire recall — cover the right column

PROMPT	ANSWER
Other name for computer architecture	Instruction Set Architecture (ISA)
Architecture in three words	The hardware–software contract
Organization in three words	The actual implementation
Four addressing modes	Immediate, direct, indirect, indexed
Two control-unit implementations	Hard-wired · micro-programmed
Definition of a logic gate	A device performing a boolean function on binary inputs, giving one binary output
AND rule / OR rule	1 only if both are 1 · 0 only if both are 0
XOR rule / XNOR rule	1 when the inputs differ · 1 when they are the same
Rows in a truth table	2^n — three inputs give 8
XOR as a sum of products	Y = A'B + AB'
De Morgan, both forms	(A·B)' = A'+B' · (A+B)' = A'·B'
The law that powers minimization	Complement: A + A' = 1
Three minimization techniques	Boolean algebra · K-map · Quine–McCluskey
Three reasons to minimize	Hardware efficiency · improved performance (less propagation delay) · lower cost and power
K-map column order	00, 01, 11, 10
K-map group sizes	Powers of 2 — 1, 2, 4, 8
Majority vote circuit, minimized	F = AB + BC + AC
SOP stands for	Sum of Products

PROMPT	ANSWER
What RTL stands for and does	Register Transfer Language — describes micro-operations between hardware registers
Meaning of the colon in RTL	Terminates a control function — the operation runs only if the condition is true
Meaning of the comma in RTL	Micro-operations execute simultaneously in one clock cycle
Four types of micro-operation	Register transfer · arithmetic · logic · control function
Definition of a micro-operation	An elementary operation on register data during a clock pulse
What the PC does	Keeps track of the next instruction to be fetched from memory
Base 8 ↔ base 2 ratio	3 : 1 ($8 = 2^3$)
Base 16 ↔ base 2 ratio	4 : 1 ($16 = 2^4$)
Decimal → other base method	Successive division; read remainders upwards
Fraction conversion method	Repeated multiplication; read integer parts downwards
Unsigned range for n bits	0 … $2^n - 1$ (256 codes for n = 8, max 255)
Two's complement from a positive	Invert every bit, then add 1
Scheme with two zeros	Signed magnitude and one's complement (+0 and −0)
Three encoding schemes	BCD · EBCDIC · ASCII
Five memory-hierarchy levels	Registers · cache · main memory · secondary · tertiary
Fastest and slowest cache level	L1 fastest and smallest · L3 largest and slowest
Three cache mappings	Direct · fully associative · set associative (N-way)
Two write policies	Write-through · write-back
Two types of locality	Temporal (a loop) · spatial (an array)
The Iron Law	CPU Time = IC × CPI × Clock Cycle Time
Performance formula	Performance = 1 / Execution Time
MIPS and FLOPS	Million Instructions Per Second · Floating Point Operations Per Second

9 Spot the error — every statement is wrong except four

#	STATEMENT	T / F	CORRECTION
1	Computer organization defines the instruction set.	F	The **architecture** defines the instruction set. Organization is the implementation — data paths, cache sizes, pipelines.

#	STATEMENT	T / F	CORRECTION
2	An 8-bit unsigned number can represent up to 256.	F	It has 256 **codes** but its maximum value is **255** — one code is spent on zero.
3	A K-map's columns are labelled 00, 01, 10, 11.	F	They are **00, 01, 11, 10** — adjacent cells must differ in exactly one bit.
4	The bubble on a gate output means NOT.	T	Correct — it is what turns AND into NAND, OR into NOR, XOR into XNOR.
5	Two's complement is formed by inverting all the bits.	F	That is **one's** complement. Two's complement is invert **then add 1**.
6	L3 cache is the fastest because it is the largest.	F	L3 is the **largest but slowest**; L1 sits on the core and is the fastest and smallest.
7	Write-back keeps cache and main memory consistent at all times.	F	That is **write-through**. Write-back is more efficient but data may be **inconsistent** between cache and main memory.
8	Spatial locality is a loop accessing the same counter.	F	That is **temporal** locality. Spatial locality is **nearby** addresses — an array or sequential structure.
9	Cache memory is non-volatile.	F	Cache is **volatile**. Secondary storage (SSD, HDD) is the non-volatile level.
10	CPU Time = Instruction Count ÷ CPI ÷ Clock Cycle Time.	F	It is a **product**: IC × CPI × clock cycle time.
11	Clock cycle time is the inverse of clock frequency.	T	Correct — 3.0 GHz gives a cycle time of $1/3.0\times10^9 \approx 0.333$ ns.
12	Octal digits are grouped in fours when converting to binary.	F	**Threes** for octal ($8 = 2^3$); fours are for hexadecimal ($16 = 2^4$).
13	Throughput is the time to complete one task from start to finish.	F	That is **response time**. Throughput is the work completed **per unit time**.
14	A minterm may be used in more than one K-map group.	T	Correct — overlapping is allowed and often gives a smaller expression, because `X + X = X`.
15	Positive numbers differ between signed magnitude, one's and two's complement.	F	Positives are **identical** in all three schemes; only the negatives differ.
16	Locality of reference is the foundation of cache memory design.	T	Correct — it is why pre-fetching a block into cache pays off.

10 Walk-in sheet — the last ten minutes

THE TEN FACTS MOST LIKELY TO BE TESTED

1. **Architecture = what/specification/ISA** · **organization = how/implementation**
2. Gates: AND 1 only if both 1 · OR 0 only if both 0 · **XOR 1 when different** · XNOR 1 when same · bubble = NOT
3. **rows = 2^n** · XOR = `A'B + AB'` · SOP = sum of products
4. Minimization rests on **A + A' = 1**; K-map columns **00 01 11 10**; groups in powers of 2; **cancel the changing variable**
5. Majority vote → **F = AB + BC + AC**
6. RTL: **← is the destination** · colon = control function · comma = same clock cycle · else = **complemented** signal
7. Micro-operations: **register transfer · arithmetic · logic · control function**
8. Bases: divide up, expand back · **3s for octal, 4s for hex** · fractions multiply downwards
9. Unsigned **0 … 2^n − 1** · **two's = one's + 1** · two's complement has one zero
10. **CPU Time = IC × CPI × Clock Cycle Time** · Performance = 1/Execution Time

NUMBERS WORTH WALKING IN WITH

- **4832_{10}** = `1001011100000`$_2$ = `11340`$_8$ = `12E0`$_{16}$
- **2025_{10}** = `11111101001`$_2$ = `3751`$_8$ = `7E9`$_{16}$
- **−56**: SM `1111000` · 1's `1000111` · 2's `1001000`
- **8-bit ranges:** unsigned 0…255 · two's complement −128…+127
- **Cache:** L1 2–64 KB · L2 256–512 KB · L3 1–8 MB

THREE MOVES IN THE FIRST FIVE MINUTES

1. Read all six questions and pick the five you will answer — leave out the one with the most unfamiliar calculation.
2. Do the calculation questions **first**, while your arithmetic is sharp.
3. Every calculation gets three lines: **formula** → **substitution** → **answer**, and every conversion gets a **check line** converting back.